UD 6. Directivas, Módulos

1 Directivas

Las directivas son elementos o atributos creados por nosotros en Angular y que añaden o modifican funcionalidad a la plantilla HTML.

1.1 Directivas de componente

En Angular, un **componente es un tipo especial de directiva** que incluye una plantilla y controla el comportamiento visual. Las directivas permiten añadir **comportamiento**, **atributos** o **estructura** al DOM.

Angular 20 mantiene estos conceptos con mejoras en el uso de componentes standalone y la composición con **hostDirectives**.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-saludo',
  standalone: true,
  template: `<h2>Hola Angular 20</h2>`
})
export class SaludoComponent {}
```

1.2 Directivas de atributo

Son directivas que **modifican la apariencia o el comportamiento** de un elemento (o componente) **sin** cambiar la estructura del DOM. Ejemplos incorporados: NgClass, NgStyle, NgModel. En Angular moderno (v20), se usan normalmente con **componentes standalone** importando las directivas que necesites.

NgClass: añadir/quitar clases CSS: Permite aplicar un conjunto de clases en función de expresiones booleanas.

```
import { Component } from '@angular/core';
import { NgClass } from '@angular/common';

@Component({
  selector: 'app-estado',
  standalone: true,
  imports: [NgClass],
  template: `
    <div [ngClass]="{ activo: isActive, error: hasError }">
      Estado: {{ isActive ? 'Activo' : 'Inactivo' }}
```

```
    </div>
  `
})
export class EstadoComponent {
  isActive = true;
  hasError = false;
}
```

NgStyle: estilos inline condicionados: Aplica estilos HTML basados en datos.

```
import { Component } from '@angular/core';
import { NgStyle } from '@angular/common';

@Component({
  selector: 'app-alerta',
  standalone: true,
  imports: [NgStyle],
  template: `
    <p [ngStyle]="{ color: level === 'high' ? 'red' : 'black' }">
      Nivel: {{ level }}
    </p>
  `
})
export class AlertaComponent {
  level: 'low' | 'high' = 'high';
}
```

1.3 Directivas estructurales

Las **directivas estructurales** modifican la **estructura del DOM**, añadiendo o eliminando elementos según condiciones o listas de datos. Se reconocen porque usan el prefijo *****. Ejemplos integrados: ***ngIf**, ***ngFor**, ***ngSwitch**

***NgSwitch**

Permite mostrar diferentes bloques según un valor.

```
@Component({
  selector: 'app-ejemplo-switch',
  standalone: true,
  template: `
    <div [ngSwitch]="rol">
      <p *ngSwitchCase="'admin'">Acceso de administrador</p>
      <p *ngSwitchCase="'user'">Acceso de usuario</p>
      <p *ngSwitchDefault>No identificado</p>
```

```
    </div>
  `
})
export class EjemploSwitchComponent {
  rol = 'admin';
}
```

***NgIf (else)**

Muestra el contenido solo si la expresión es verdadera.

```
@Component({
  selector: 'app-ejemplo-if',
  standalone: true,
  imports: [],
  template: `
    <p *ngIf="activo">El usuario está activo</p>
  `
})
export class EjemploIfComponent {
  activo = true;
}
```

***ngfor**

Itera sobre una colección y crea un elemento por ítem.

```
@Component({
  selector: 'app-ejemplo-for',
  standalone: true,
  imports: [],
  template: `
    <ul>
      <li *ngFor="let item of items; let i = index">
        {{ i }} - {{ item }}
      </li>
    </ul>
  `
})
export class EjemploForComponent {
  items = ['Angular', 'Directivas', 'Estructurales'];
}
```

Creando una directiva estructural

Una **directiva estructural** modifica la **estructura del DOM**, mostrando u ocultando partes de la vista. Para crear una propia, se sigue el mismo patrón que Angular usa internamente con *ngIf o *ngFor:

- Definir un selector con **[nombre]**
- Usar **TemplateRef** (plantilla a renderizar)
- Usar **ViewContainerRef** (contenedor que inserta/quita vistas)
- Implementar un **Input** que controle el renderizado

Paso 1: Creando la directiva

```
import { Directive, Input, TemplateRef, ViewContainerRef } from
'@angular/core';

@Directive({
  selector: '[appSi]',
  standalone: true
})
export class SiDirective {

  constructor(
    private template: TemplateRef<any>,
    private view: ViewContainerRef
  ) {}

  @Input() set appSi(condicion: boolean) {
    this.view.clear();                // Limpia el contenido previo
    if (condicion) {
      this.view.createEmbeddedView(this.template); // Inserta la
plantilla
    }
  }
}
```

Paso 2: Usándola en un componente

```
<p *appSi="mostrar">Este texto aparece solo si 'mostrar' es
true</p>
```

ng-container: Permite combinar varias estructurales (*ngIf, *ngFor, *ngSwitch, etc.)

```
<ng-container *ngIf="items.length > 0">
  <ul>
    <li *ngFor="let item of items">
      {{ item }}
    </li>
  </ul>
</ng-container>
```

2 Módulos

2.1 Array de inicialización (Bootstrap)

En Angular actual, se aconseja **arrancar la app sin NgModule**, con un **componente standalone** y bootstrapApplication en main.ts. Ahí mismo puedes registrar **providers** globales (Router, HttpClient, etc.)

```
// main.ts (standalone)
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter([]),
    provideHttpClient()
  ]
}).catch(err => console.error(err));
```

2.2 Array declarations

En Angular 20, **los componentes standalone NO usan NgModules**, por lo que el array declarations **ya no se utiliza**.

En standalone:

- **Cada componente se declara a sí mismo** con standalone: true.
- Ya **no se registran** componentes, directivas ni pipes dentro de un NgModule.
- En lugar de declarations, standalone usa el array **imports** en el propio componente, indicando qué directivas, pipes u otros componentes necesita

App.component.ts

```
import { Component } from '@angular/core';
import { HeaderComponent } from './header.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [HeaderComponent],   // Sustituye a "declarations"
  template: `
    <app-header></app-header>
    <h1>App Standalone Angular 20</h1>
  `
})
export class AppComponent {}
```

header.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-header',
  standalone: true,
  template: `<header>Mi encabezado</header>`
})
export class HeaderComponent {}
```

main.ts

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent);
```

2.3 Array imports

Sirve para **incluir otros componentes, directivas o pipes standalone**, así como providers funcionales (Router, HttpClient…)

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { HeaderComponent } from './header.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [
    CommonModule,      // directivas estándar (ngIf, ngFor…)
    HeaderComponent    // componente standalone
```

```
  ],
  template: `
    <app-header></app-header>
    <p>Standalone Angular 20</p>
  `
})
export class AppComponent {}
```

2.4 Array providers

Los providers pueden declararse:

A nivel global (en el arranque, como sustituto de los providers en un NgModule): Usando bootstrapApplication(), que permite registrar providers para toda la app

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter([]),
    provideHttpClient()
  ]
});
```

A nivel de componente: Un componente standalone puede registrar providers propios.

```
@Component({
  selector: 'app-panel',
  standalone: true,
  template: `Panel`,
  providers: [MiServicio]
})
export class PanelComponent {}
```

3 Dividiendo nuestra aplicación en módulos

3.1 División por características

Para organizar una aplicación en módulos hay que seguir ciertas reglas. Algunas de las que se recomiendan son las siguientes:

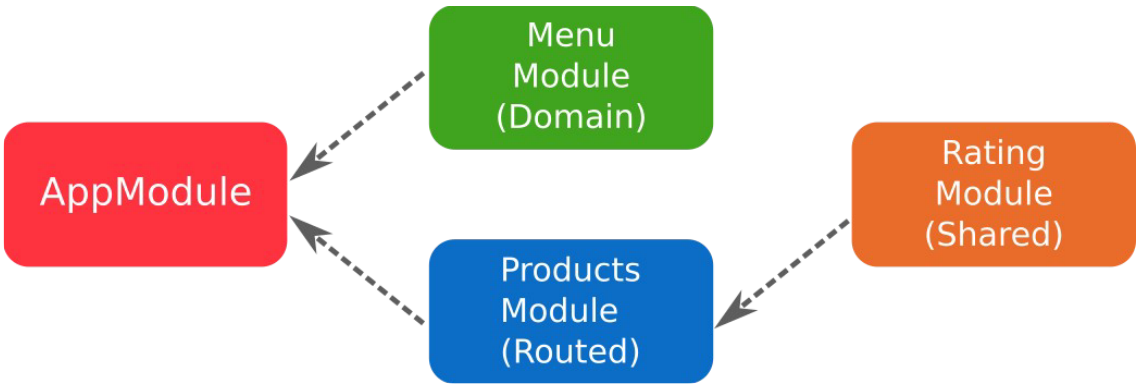
• **Módulos de dominio**: módulos que se importan en el módulo de la aplicación. Sirven simplemente para separar código del módulo principal (aquí no meteríamos componentes que representen una ruta). Un ejemplo, sería un módulo de **Menú**.

• **Módulos de sección (Routed)**: módulos que representan una sección de nuestra aplicación (un conjunto de rutas relacionadas entre sí). Gestionan un conjunto de páginas relacionadas. Por ejemplo un módulo de **Productos** podría contener los componentes ProductsList, ProductDetail, ProductEdit, .…

• **Módulos de rutas (Routing)**: Modulo asociados con un "módulo de sección" . Sirve para separar la gestión de las rutas.

• **Módulo de servicio**: Modulo o módulos que declaran una serie de servicios. Solo se importan desde el módulo de la aplicación (AppModule) y tiene el mismo efecto que simplemente declarar los servicios ahí (sirve para separar código). Normalmente no se declaran componentes aquí. Ejemplo →HttpClientModule de Angular.

• **Módulos de componentes**: módulos que exportan componentes, directivas y pipes que pueden ser usados a su vez en diferentes módulos de la aplicación. Por ejemplo el componente **StarRatingComponent** podría ir en uno de estos módulos (RatingModule).



3.2 Creando módulos de dominio

Un "módulo de dominio" significa **organizar el código por áreas funcionales** usando:

- **Carpetas de dominio**
- **Componentes standalone**
- **Servicios**
- **Rutas propias del dominio**

Estructura recomendada de un dominio

```
src/app/products/
   product-list/
         product-list.component.ts
   product-detail/
         product-detail.component.ts
   products.routes.ts
   products.service.ts
   index.ts (opcional)
```

product-list.component.ts

```
import { Component } from '@angular/core';
import { NgFor } from '@angular/common';

@Component({
  selector: 'product-list',
  standalone: true,
  imports: [NgFor],
  template: `
    <h2>Productos</h2>
    <ul>
      <li *ngFor="let p of products">{{ p }}</li>
    </ul>
  `
})
export class ProductListComponent {
  products = ['Teclado', 'Ratón', 'Monitor'];
}
```

products.routes.ts

```
import { Routes } from '@angular/router';
import { ProductListComponent } from './product-list/product-
list.component';

export const PRODUCTS_ROUTES: Routes = [
  { path: '', component: ProductListComponent }
```

```
];
```

products.service.ts

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class ProductsService {
  getAll() {
    return ['Teclado', 'Ratón', 'Monitor'];
  }
}
```

app.routes.ts

```
import { Routes } from '@angular/router';
import { PRODUCTS_ROUTES } from './products/products.routes';

export const routes: Routes = [
  {
    path: 'products',
    children: PRODUCTS_ROUTES
  }
];
```

main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app.routes';

bootstrapApplication(AppComponent, {
  providers: [provideRouter(routes)]
});
```

3.3 Creando módulos de sección (Routed)

En standalone, un módulo de sección routed es simplemente:

- Una **carpeta** que agrupa la funcionalidad (ej. dashboard/, products/, admin/)
- Uno o varios **componentes standalone**
- Un archivo de **rutas** de esa sección
- (Opcional) servicios o barrel file

```
src/app/dashboard/
            dashboard.component.ts
            dashboard.routes.ts
   index.ts (opcional)
```

dashboard.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'dashboard-page',
  standalone: true,
  template: `
    <h2>Dashboard</h2>
    <p>Bienvenido al panel</p>
  `
})
export class DashboardComponent {}
```

dashboard.routes.ts

```
import { Routes } from '@angular/router';
import { DashboardComponent } from './dashboard.component';

export const DASHBOARD_ROUTES: Routes = [
  { path: '', component: DashboardComponent }
];
```

app.routes.ts

```
import { Routes } from '@angular/router';
import { DASHBOARD_ROUTES } from
'./dashboard/dashboard.routes';

export const routes: Routes = [
  {
    path: 'dashboard',
    children: DASHBOARD_ROUTES
  }
];
```

main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app.routes';

bootstrapApplication(AppComponent, {
  providers: [provideRouter(routes)]
});
```

3.4 Creando módulos de rutas (Routing)

Un "módulo de rutas" en standalone **no es un NgModule**, sino simplemente **un archivo de definición de rutas**, junto con los componentes standalone que las atienden.

app.routes.ts

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { DashboardComponent } from
'./dashboard/dashboard.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'dashboard', component: DashboardComponent }
];
```

home.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'home-page',
  standalone: true,
  template: `<h2>Home</h2>`
})
export class HomeComponent {}
```

dashboard.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'dashboard-page',
  standalone: true,
  template: `<h2>Dashboard</h2>`
})
export class DashboardComponent {}
```

main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { AppComponent } from './app/app.component';
import { routes } from './app.routes';

bootstrapApplication(AppComponent, {
  providers: [provideRouter(routes)]
});
```

app.component.ts

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  template: `
    <h1>Mi Aplicación</h1>
    <router-outlet></router-outlet>
  `
})
export class AppComponent {}
```

dashboard.routes.ts

```
import { Routes } from '@angular/router';
import { DashboardComponent } from './dashboard.component';

export const DASHBOARD_ROUTES: Routes = [
  { path: '', component: DashboardComponent }
];
```

3.5 Creando módulos de servicio

n Angular, un **servicio** es una clase que encapsula lógica reutilizable (peticiones HTTP, gestión de datos, utilidades…).

Para organizar mejor la aplicación, estos servicios suelen agruparse dentro de **módulos propios**. Puedes trabajar con **standalone** o con **NgModules**

Crear un servicio

```
ng generate service services/usuario

Esto crea:

src/app/services/
    usuario.service.ts
```

Ejemplo de servicio simple podría ser

usuario.service.ts

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class UsuarioService {

  getUsuario() {
    return { nombre: 'Paco', rol: 'Profesor' };
  }
}
```

Crear un módulo para agrupar servicios

Aunque Angular 20 permite servicios standalone, sigue siendo muy útil crear un **módulo de servicios** para organizar proyectos grandes

```
ng generate module services

Esto crea:

src/app/services/
    services.module.ts
```

Este módulo **centraliza** todos los servicios que quieras registrar

services.module.ts

```
import { NgModule } from '@angular/core';
import { UsuarioService } from './usuario.service';

@NgModule({
  providers: [UsuarioService]
})
export class ServicesModule {}
```

En Angular 20 con standalone, lo más habitual es importarlo en app.config.ts

app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { ServicesModule } from './services/services.module';

export const appConfig: ApplicationConfig = {
```

```
  providers: [
    provideRouter([]),
    importProvidersFrom(ServicesModule)
  ]
};
```

Para usar ese servicio dentro del componente **HomeComponent**

src/app/home/home.component.ts

```
import { Component } from '@angular/core';
import { UsuarioService } from '../services/usuario.service';

@Component({
  selector: 'app-home',
  standalone: true,
  template: `
    <h2>Usuario: {{ usuario.nombre }}</h2>
    <p>Rol: {{ usuario.rol }}</p>
  `
})
export class HomeComponent {

  usuario = this.usuarioService.getUsuario();

  constructor(private usuarioService: UsuarioService) {}
}
```

3.6 Creando Módulos de componentes

Aunque Angular 20 trabaja principalmente con **componentes standalone**, sigue siendo habitual crear **módulos de componentes** cuando quieres:

- Agrupar componentes relacionados
- Compartir componentes entre distintas partes de la app
- Organizar mejor los proyectos grandes
- Reutilizar componentes en múltiples módulos o features

Crear un componente dentro del módulo

```
ng generate component shared/card

Esto genera:
src/app/shared/card/
    card.component.ts
    card.component.html
    card.component.css
```

Declarar y exportar el componente en el módulo

shared.module.ts

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { CardComponent } from './card/card.component';

@NgModule({
    declarations: [CardComponent],
    imports: [CommonModule],
    exports: [CardComponent] // ← Muy importante
 })

export class SharedModule {}
```

Usar el módulo en un componente standalone

En Angular 20, un componente standalone puede importar módulos tradicionales.

home.component.ts

```
import { Component } from '@angular/core';
import { SharedModule } from '../shared/shared.module';

@Component({
    selector: 'app-home',
    standalone: true,
    imports: [SharedModule],
    template: `
      <h2>Página principal</h2>
      <app-card></app-card>
   `
})

export class HomeComponent {}
```

3.7 Carga en diferido (lazy-loading) de módulos

El **lazy-loading** permite cargar partes de la aplicación **solo cuando el usuario las necesita**, reduciendo el tamaño inicial y acelerando la carga.

- **Lazy-loading de un componente standalone**

Crear el componente

```
ng generate component pages/ajustes –standalone

Esto nos genera:

src/app/pages/ajustes/ajustes.component.ts
```

Configurar la ruta con carga diferida
app.routes.ts

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'ajustes',
    loadComponent: () =>
      import('./pages/ajustes/ajustes.component')
          .then(m => m.AjustesComponent)
  }
];
```

- **Lazy-loading de un módulo (NgModule)**

Crear un módulo con rutas internas

```
ng generate module admin --routing

Esto crea:

src/app/admin/admin.module.ts
src/app/admin/admin-routing.module.ts
```

Configurar lazy-loading en el router principal

app.routes.ts

```
export const routes: Routes = [
  {
    path: 'admin',
    loadChildren: () =>
      import('./admin/admin.module')
          .then(m => m.AdminModule)
  }
];
```

3.8 Definiendo la estrategia de carga en diferido (preloading)

El **preloading** permite que Angular cargue en segundo plano los módulos o componentes que están configurados como *lazy-loaded*, **después** de que la aplicación inicial ya esté lista.

Resultado:

- La app arranca rápido (lazy-loading)
- Pero las rutas diferidas se cargan anticipadamente (preloading)
- Mejor experiencia sin retrasos al navegar

Configurar rutas con lazy-loading

app.routes.ts

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'admin',
    loadChildren: () =>
      import('./admin/admin.module').then(m => m.AdminModule)
  },
  {
    path: 'productos',
    loadChildren: () =>
    import('./productos/productos.module').then(m => m.ProductosModule)
  }
];
```

Preloading selectivo (solo algunas rutas): Puedes decidir qué rutas se precargan usando la propiedad data.

```
{
  path: 'admin',
  loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule),
  data: { preload: true }
}
```

Crear una estrategia personalizada

preload-selected.strategy.ts

```
import { PreloadingStrategy, Route } from '@angular/router';
import { Observable, of } from 'rxjs';
 export class PreloadSelectedStrategy implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>): Observable<any> {
   return route.data?.['preload'] ¿ load() : of(null);
 }
}
```

app.config.ts

```
import { PreloadSelectedStrategy } from './preload-selected.strategy';

export const appConfig: ApplicationConfig = {
  providers: [
  provideRouter(
     routes,
     withPreloading(PreloadSelectedStrategy)
  )
 ]
};
```